

COMPLAINT MANAGEMENT SYSTEM

ID : 141466

Name: NandhaKumar V

College: Agurchand Manmull Jain College

GitHub Repository: https://github.com/Nandhakumar-007/Complaint_Management_MySql_project

PROJECT OVERVIEW :

The Complaint Management System is designed to manage customer complaints efficiently by storing user details, complaint records, employee assignments, complaint status updates, complaint history, and feedback.

The system helps track complaint lifecycles from registration to resolution. It also supports reporting, employee performance analysis, and customer satisfaction analysis using SQL queries.

Main features of the system include:

Complaint registration.

Complaint assignment.

Status tracking.

History logging.

Feedback collection.

Performance reporting.

Database Schema :

```
// create database complaint_management;  
// use complaint_management;
```

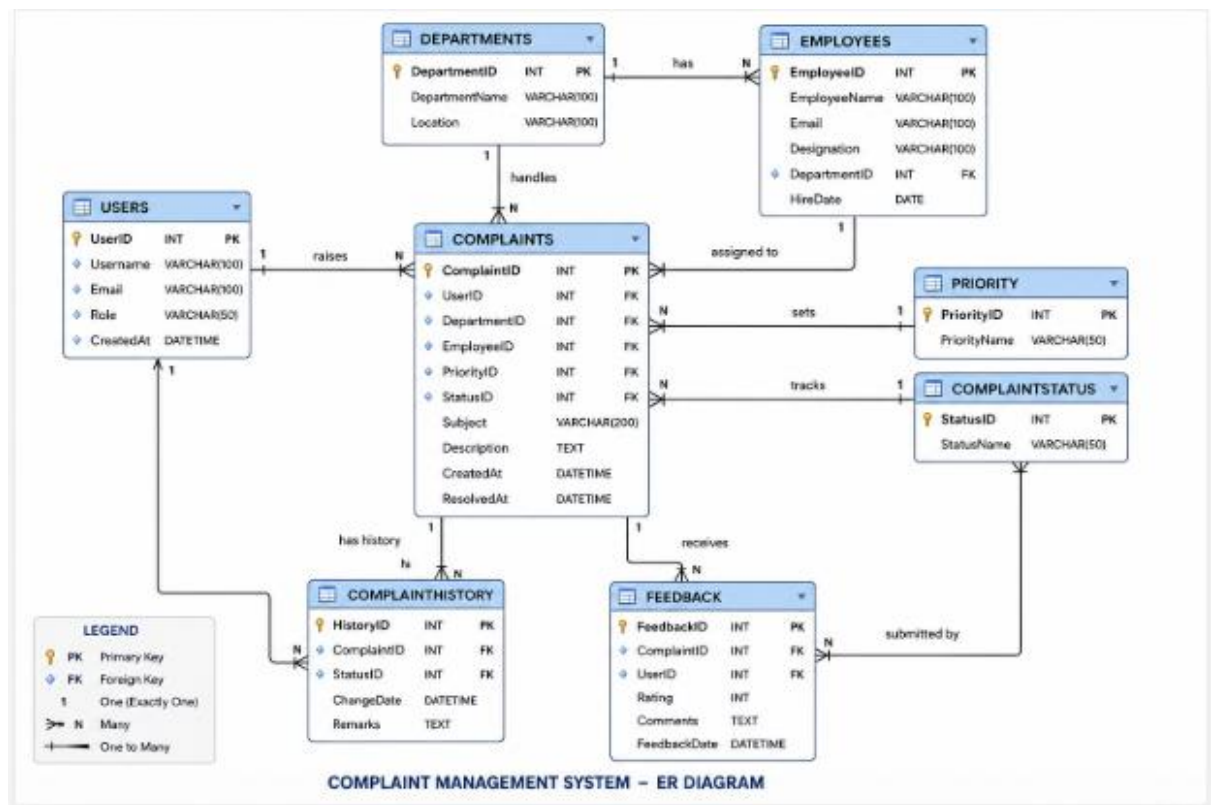
Main Tables :

- Users
- Employees
- Departments
- Priority
- ComplaintStatus
- Complaints
- ComplaintHistory
- Feedback

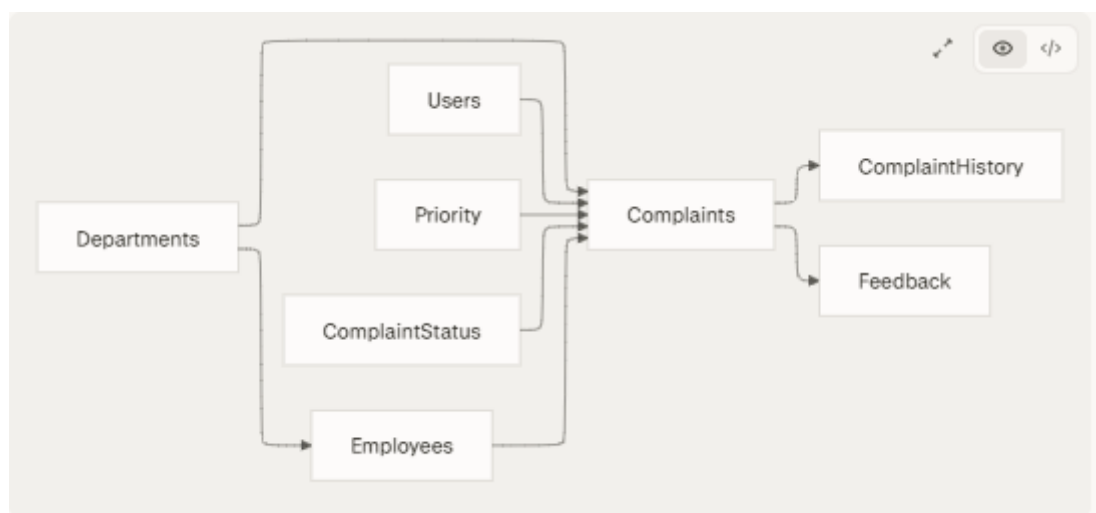
Relationships :

- A user can raise many complaints.
- A complaint belongs to one department.
- A complaint can be assigned to one employee.
- A complaint can have many history records.
- A complaint can have one feedback record.

ER Diagram :



Relationship Diagram :



Beginner SQL Questions & Queries :

-- Q1: List all departments.

```
SELECT * FROM Departments;
```

-- Q2: List all users with role 'Customer'.

```
SELECT * FROM Users WHERE Role = 'Customer';
```

-- Q3: Show all complaints with PriorityID = 4 (Critical).

```
SELECT * FROM Complaints WHERE PriorityID = 4;
```

-- Q4: List all employees working in DepartmentID = 1.

```
SELECT * FROM Employees WHERE DepartmentID = 1;
```

-- Q5: Show all complaint statuses available in the system.

```
SELECT * FROM ComplaintStatus;
```

-- Q6: Find all complaints that are still unresolved (ResolvedAt is NULL).

```
SELECT * FROM Complaints WHERE ResolvedAt IS NULL;
```

-- Q7: List the names and emails of all employees.

```
SELECT EmployeeName, Email FROM Employees;
```

-- Q8: Show all feedback with a rating of 5.

```
SELECT * FROM Feedback WHERE Rating = 5;
```

-- Q9: List all complaints sorted by CreatedAt in descending order.

```
SELECT * FROM Complaints ORDER BY CreatedAt DESC;
```

-- Q10: Find the total number of users in the system.

```
SELECT COUNT(*) AS TotalUsers FROM Users;
```

-- Q11: Find the total number of complaints registered.

```
SELECT COUNT(*) AS TotalComplaints FROM Complaints;
```

-- Q12: List all employees hired after 2022-01-01.

```
SELECT * FROM Employees WHERE HireDate > '2022-01-01';
```

-- Q13: Show all distinct roles present in the Users table.

```
SELECT DISTINCT Role FROM Users;
```

-- Q14: Find all complaints with the subject containing the word 'Laptop'.

```
SELECT * FROM Complaints WHERE Subject LIKE '%Laptop%';
```

-- Q15: List the first 10 complaints by ComplaintID.

```
SELECT * FROM Complaints ORDER BY ComplaintID ASC LIMIT 10;
```

-- Q16: Show all users created between '2024-01-01' and '2024-12-31'.

```
SELECT * FROM Users WHERE CreatedAt BETWEEN '2024-01-01' AND '2024-12-31';
```

-- Q17: Find all feedback comments that mention the word 'satisfied'.

```
SELECT * FROM Feedback WHERE Comments LIKE '%satisfied%';
```

-- Q18: List all department names along with their locations.

```
SELECT DepartmentName, Location FROM Departments;
```

-- Q19: Find the highest rating given in Feedback.

```
SELECT MAX(Rating) AS HighestRating FROM Feedback;
```

-- Q20: Find the lowest priority level (PriorityID) used in Complaints.

```
SELECT MIN(PriorityID) AS LowestPriorityUsed FROM Complaints;
```

Intermediate SQL Questions & Queries

-- Q1: List all complaints along with the username of the person who raised them.

```
SELECT c.ComplaintID, c.Subject, u.Username  
FROM Complaints c  
JOIN Users u ON c.UserID = u.UserID;
```

-- Q2: Show all complaints with their department name.

```
SELECT c.ComplaintID, c.Subject, d.DepartmentName  
FROM Complaints c  
JOIN Departments d ON c.DepartmentID = d.DepartmentID;
```

-- Q3: List all complaints with their assigned employee's name and designation.

```
SELECT c.ComplaintID, e.EmployeeName, e.Designation  
FROM Complaints c  
JOIN Employees e ON c.EmployeeID = e.EmployeeID;
```

-- Q4: Show complaints along with their priority name and status name.

```
SELECT c.ComplaintID, p.PriorityName, s.StatusName  
FROM Complaints c  
JOIN Priority p ON c.PriorityID = p.PriorityID  
JOIN ComplaintStatus s ON c.StatusID = s.StatusID;
```

-- Q5: Count the number of complaints per department.

```
SELECT d.DepartmentName, COUNT(c.ComplaintID) AS TotalComplaints
FROM Departments d
LEFT JOIN Complaints c ON d.DepartmentID = c.DepartmentID
GROUP BY d.DepartmentName;
```

-- Q6: Count the number of complaints per status.

```
SELECT s.StatusName, COUNT(c.ComplaintID) AS TotalComplaints
FROM ComplaintStatus s
LEFT JOIN Complaints c ON s.StatusID = c.StatusID
GROUP BY s.StatusName;
```

-- Q7: Find the average feedback rating per department.

```
SELECT d.DepartmentName, ROUND(AVG(f.Rating), 2) AS AvgRating
FROM Feedback f
JOIN Complaints c ON f.ComplaintID = c.ComplaintID
JOIN Departments d ON c.DepartmentID = d.DepartmentID
GROUP BY d.DepartmentName;
```

-- Q8: List departments having more than 15 complaints.

```
SELECT d.DepartmentName, COUNT(c.ComplaintID) AS TotalComplaints
FROM Departments d
JOIN Complaints c ON d.DepartmentID = c.DepartmentID
GROUP BY d.DepartmentName
HAVING COUNT(c.ComplaintID) > 15;
```

-- Q9: Find the number of complaints handled by each employee.

```
SELECT e.EmployeeName, COUNT(c.ComplaintID) AS ComplaintsHandled
FROM Employees e
LEFT JOIN Complaints c ON e.EmployeeID = c.EmployeeID
GROUP BY e.EmployeeName
ORDER BY ComplaintsHandled DESC;
```

-- Q10: List all complaints along with the number of history entries each has.

```
SELECT c.ComplaintID, c.Subject, COUNT(h.HistoryID) AS StatusChanges
FROM Complaints c
LEFT JOIN ComplaintHistory h ON c.ComplaintID = h.ComplaintID
GROUP BY c.ComplaintID, c.Subject;
```

-- Q11: Find all users who have never submitted feedback.

```
SELECT u.UserID, u.Username
FROM Users u
WHERE u.UserID NOT IN (SELECT DISTINCT UserID FROM Feedback);
```

-- Q12: List the top 5 employees with the most assigned complaints.

```
SELECT e.EmployeeName, COUNT(c.ComplaintID) AS TotalAssigned
FROM Employees e
JOIN Complaints c ON e.EmployeeID = c.EmployeeID
GROUP BY e.EmployeeName
ORDER BY TotalAssigned DESC
LIMIT 5;
```


-- Q13: Find the average resolution time (in days) for resolved complaints.

```
SELECT ROUND(AVG(DATEDIFF(ResolvedAt, CreatedAt)), 2) AS AvgResolutionDays  
FROM Complaints  
WHERE ResolvedAt IS NOT NULL;
```

-- Q14: List all complaints that received a feedback rating below 3, with department and employee names.

```
SELECT c.ComplaintID, d.DepartmentName, e.EmployeeName, f.Rating  
FROM Feedback f  
JOIN Complaints c ON f.ComplaintID = c.ComplaintID  
JOIN Departments d ON c.DepartmentID = d.DepartmentID  
JOIN Employees e ON c.EmployeeID = e.EmployeeID  
WHERE f.Rating < 3;
```

-- Q15: Show the count of complaints raised by each user, only for users with more than 2 complaints.

```
SELECT u.Username, COUNT(c.ComplaintID) AS ComplaintCount  
FROM Users u  
JOIN Complaints c ON u.UserID = c.UserID  
GROUP BY u.Username  
HAVING COUNT(c.ComplaintID) > 2;
```

-- Q16: Find the department with the highest number of critical priority complaints.

```
SELECT d.DepartmentName, COUNT(c.ComplaintID) AS CriticalComplaints  
FROM Complaints c  
JOIN Departments d ON c.DepartmentID = d.DepartmentID  
WHERE c.PriorityID = 4
```

```
GROUP BY d.DepartmentName
ORDER BY CriticalComplaints DESC
LIMIT 1;
```

-- Q17: List complaint history entries along with the complaint subject and the status name.

```
SELECT h.HistoryID, c.Subject, s.StatusName, h.ChangeDate, h.Remarks
FROM ComplaintHistory h
JOIN Complaints c ON h.ComplaintID = c.ComplaintID
JOIN ComplaintStatus s ON h.StatusID = s.StatusID;
```

-- Q18: Find the average number of complaints assigned per employee, by department.

```
SELECT d.DepartmentName, COUNT(c.ComplaintID) / COUNT(DISTINCT e.EmployeeID) AS
AvgComplaintsPerEmployee
FROM Departments d
JOIN Employees e ON d.DepartmentID = e.DepartmentID
LEFT JOIN Complaints c ON e.EmployeeID = c.EmployeeID
GROUP BY d.DepartmentName;
```

-- Q19: List all complaints raised in the last 30 days with user and status details.

```
SELECT c.ComplaintID, u.Username, s.StatusName, c.CreatedAt
FROM Complaints c
JOIN Users u ON c.UserID = u.UserID
JOIN ComplaintStatus s ON c.StatusID = s.StatusID
WHERE c.CreatedAt >= DATE_SUB(CURDATE(), INTERVAL 30 DAY);
```

-- Q20: Find the total feedback count and average rating for each employee.

```
SELECT e.EmployeeName, COUNT(f.FeedbackID) AS TotalFeedback, ROUND(AVG(f.Rating),
2) AS AvgRating
FROM Employees e
```

JOIN Complaints c ON e.EmployeeID = c.EmployeeID

JOIN Feedback f ON c.ComplaintID = f.ComplaintID

GROUP BY e.EmployeeName

ORDER BY AvgRating DESC;

Advanced SQL Questions & Queries

-- Q1: Rank employees within each department by number of complaints handled (window function).

```
SELECT e.EmployeeName, e.DepartmentID, COUNT(c.ComplaintID) AS TotalComplaints,  
       RANK() OVER (PARTITION BY e.DepartmentID ORDER BY COUNT(c.ComplaintID) DESC)  
AS DeptRank
```

FROM Employees e

LEFT JOIN Complaints c ON e.EmployeeID = c.EmployeeID

GROUP BY e.EmployeeID, e.EmployeeName, e.DepartmentID;

-- Q2: Find the top-rated employee (by average feedback rating) in each department using a CTE.

WITH EmployeeRatings AS (

```
    SELECT e.EmployeeID, e.EmployeeName, e.DepartmentID, AVG(f.Rating) AS AvgRating
```

```
    FROM Employees e
```

```
    JOIN Complaints c ON e.EmployeeID = c.EmployeeID
```

```
    JOIN Feedback f ON c.ComplaintID = f.ComplaintID
```

```
    GROUP BY e.EmployeeID, e.EmployeeName, e.DepartmentID
```

),

Ranked AS (

```
    SELECT *, RANK() OVER (PARTITION BY DepartmentID ORDER BY AvgRating DESC) AS rnk
```

```
    FROM EmployeeRatings
```

```
)  
  
SELECT DepartmentID, EmployeeName, AvgRating  
  
FROM Ranked  
  
WHERE rnk = 1;
```

-- Q3: Find the running total of complaints raised per month.

```
SELECT DATE_FORMAT(CreatedAt, '%Y-%m') AS Month,  
  
       COUNT(*) AS MonthlyComplaints,  
  
       SUM(COUNT(*)) OVER (ORDER BY DATE_FORMAT(CreatedAt, '%Y-%m')) AS  
RunningTotal  
  
FROM Complaints  
  
GROUP BY DATE_FORMAT(CreatedAt, '%Y-%m')  
  
ORDER BY Month;
```

-- Q4: Find complaints that took longer than the average resolution time to resolve.

```
SELECT ComplaintID, Subject, DATEDIFF(ResolvedAt, CreatedAt) AS ResolutionDays  
  
FROM Complaints  
  
WHERE ResolvedAt IS NOT NULL  
  
AND DATEDIFF(ResolvedAt, CreatedAt) > (  
  
    SELECT AVG(DATEDIFF(ResolvedAt, CreatedAt))  
  
    FROM Complaints  
  
    WHERE ResolvedAt IS NOT NULL  
  
);
```

-- Q5: For each department, show the percentage of complaints that are 'Closed'.

```
SELECT d.DepartmentName,  
  
       ROUND(100.0 * SUM(CASE WHEN s.StatusName = 'Closed' THEN 1 ELSE 0 END) /  
COUNT(c.ComplaintID), 2) AS PctClosed  
  
FROM Departments d
```

JOIN Complaints c ON d.DepartmentID = c.DepartmentID

JOIN ComplaintStatus s ON c.StatusID = s.StatusID

GROUP BY d.DepartmentName;

-- Q6: Find the second highest average feedback rating among employees (without LIMIT/OFFSET tricks).

SELECT MAX(AvgRating) AS SecondHighestAvg

FROM (

SELECT e.EmployeeID, AVG(f.Rating) AS AvgRating

FROM Employees e

JOIN Complaints c ON e.EmployeeID = c.EmployeeID

JOIN Feedback f ON c.ComplaintID = f.ComplaintID

GROUP BY e.EmployeeID

) AS EmpAvg

WHERE AvgRating < (

SELECT MAX(AvgRating) FROM (

SELECT e.EmployeeID, AVG(f.Rating) AS AvgRating

FROM Employees e

JOIN Complaints c ON e.EmployeeID = c.EmployeeID

JOIN Feedback f ON c.ComplaintID = f.ComplaintID

GROUP BY e.EmployeeID

) AS Sub

);

-- Q7: Find complaints whose history shows a 'Reopened' status entry.

SELECT DISTINCT c.ComplaintID, c.Subject, u.Username

FROM Complaints c

JOIN ComplaintHistory h ON c.ComplaintID = h.ComplaintID

```
JOIN ComplaintStatus s ON h.StatusID = s.StatusID
```

```
JOIN Users u ON c.UserID = u.UserID
```

```
WHERE s.StatusName = 'Reopened';
```

-- Q8: For each priority level, find the average resolution time and compare it to the overall average using a CTE.

```
WITH PriorityAvg AS (
```

```
    SELECT p.PriorityName, AVG(DATEDIFF(c.ResolvedAt, c.CreatedAt)) AS AvgDays
```

```
    FROM Complaints c
```

```
    JOIN Priority p ON c.PriorityID = p.PriorityID
```

```
    WHERE c.ResolvedAt IS NOT NULL
```

```
    GROUP BY p.PriorityName
```

```
),
```

```
OverallAvg AS (
```

```
    SELECT AVG(DATEDIFF(ResolvedAt, CreatedAt)) AS OverallDays
```

```
    FROM Complaints
```

```
    WHERE ResolvedAt IS NOT NULL
```

```
)
```

```
SELECT PriorityName, ROUND(AvgDays, 2) AS AvgDays,
```

```
    ROUND((SELECT OverallDays FROM OverallAvg), 2) AS OverallAvgDays,
```

```
    ROUND(AvgDays - (SELECT OverallDays FROM OverallAvg), 2) AS Difference
```

```
FROM PriorityAvg;
```

-- Q9: Find the employee(s) who have never received a feedback rating below 4 (consistently good performers).

```
SELECT e.EmployeeName
```

```
FROM Employees e
```

```
JOIN Complaints c ON e.EmployeeID = c.EmployeeID
```

```
JOIN Feedback f ON c.ComplaintID = f.ComplaintID
```

GROUP BY e.EmployeeID, e.EmployeeName

HAVING MIN(f.Rating) >= 4;

-- Q10: Create a monthly summary report: complaints raised, complaints resolved, and average rating, using multiple CTEs.

WITH Raised AS (

SELECT DATE_FORMAT(CreatedAt, '%Y-%m') AS Month, COUNT(*) AS RaisedCount

FROM Complaints

GROUP BY DATE_FORMAT(CreatedAt, '%Y-%m')

),

Resolved AS (

SELECT DATE_FORMAT(ResolvedAt, '%Y-%m') AS Month, COUNT(*) AS ResolvedCount

FROM Complaints

WHERE ResolvedAt IS NOT NULL

GROUP BY DATE_FORMAT(ResolvedAt, '%Y-%m')

),

Ratings AS (

SELECT DATE_FORMAT(f.FeedbackDate, '%Y-%m') AS Month, ROUND(AVG(f.Rating), 2) AS AvgRating

FROM Feedback f

GROUP BY DATE_FORMAT(f.FeedbackDate, '%Y-%m')

)

SELECT r.Month, r.RaisedCount,

COALESCE(res.ResolvedCount, 0) AS ResolvedCount,

COALESCE(rt.AvgRating, 0) AS AvgRating

FROM Raised r

LEFT JOIN Resolved res ON r.Month = res.Month

LEFT JOIN Ratings rt ON r.Month = rt.Month

ORDER BY r.Month;

Stored Procedures

Duplicate Insurance Prevention :

```
drop procedure if exists DIP;

delimiter //

create procedure DIP
(IN p_InsuranceID int,
IN p_PatientID int,
IN p_Disease text,
IN p_Provider text)
begin
declare CountInsurance int;
select count(i.insuranceID) into CountInsurance
from insurance i inner join appointments a on i.patientID=a.patientID
inner join diagnosis d on a.appointmentID=d.appointmentID
where i.patientID=p_PatientID and d.disease=p_Disease;
if countInsurance>1 then
signal SQLState '45000'
SET MESSAGE_TEXT = 'Prohibited: Duplicate Entry';
else
insert into insurance(insuranceID,patientID,Provider)
values(p_InsuranceID,p_PatientID,p_Provider);
end if;
end//

delimiter ;

call DIP(1,3597,'Allergy','LIC');

show procedure status like 'DIP';
```


Room Booking :

```
drop procedure if exists ARB;

delimiter //

create procedure RB(in p_roomID int)

begin

declare rStatus text;

declare rID int;

select roomID, AvailabilityStatus into rID, rStatus from rooms where roomID=p_roomID;

if rID is null then

signal sqlstate '45000'

set MESSAGE_TEXT ='Room Not Available';

elseif rStatus='Available' then

update rooms set AvailabilityStatus='Occupied' where roomID=p_roomID;

else

signal SQLState '45000'

SET MESSAGE_TEXT = 'Prohibited: Duplicate Entry';

end if;

end //

delimiter ;
```

Triggers

Payment Status Automation :

```
delimiter //

create trigger paymentAutomation

after insert on payments

for each row

begin

declare totalAmt int;

declare paidAmt int;

select totalAmount into totalAmt

from bills where billID=new.BillID;

select coalesce(sum(amountPaid),0) into paidAmt

from payments where billID=new.BillID;

if TotalAmt = paidamt then

update bills set status="Paid" where billId=new.BillID;

elseif TotalAmt<paidAmt then

update bills set status="suspicious" where billId=new.BillID;

else

update bills set status="Partially paid" where billId=new.BillID;

end if;

end //

delimiter ;
```

CONCLUSION :

The Complaint Management System is a complete SQL project for managing complaints, employees, departments, status updates, and feedback. It demonstrates practical use of SQL joins, aggregations, subqueries, CTEs, window functions, stored procedures, and triggers.